

Data contamination evaluation in fine-tuning

Martina Lupini

Andrea Andreoli

Pierfrancesco Lijoi

dept. Computer Engineering, University of Rome Tor Vergata
Rome, Italy

Abstract—In questo articolo analizzeremo varie metriche per la rilevazione di data contamination nei modelli LLM. In seguito ci focalizzeremo nel caso di contaminazione durante la fase di fine-tuning e proporremo una soluzione innovativa.

I. INTRODUZIONE

Gli LLM o Large Language Models sono modelli utilizzati nell'ambito del NLP per la comprensione e generazione di linguaggio. Tipicamente vengono pre-addestrati su grandi corpus di testo e poi sottoposti a fine-tuning su dataset specifici in modo da specializzare il modello su un certo task. Negli ultimi due anni sono nate delle perplessità circa la validità della valutazione degli LLM a causa del problema della *data contamination*. Data contamination fa riferimento al fenomeno in cui istanze dei dati di testing sono trovate anche all'interno dei dati di training. Nel corso degli anni, l'uso di una quantità sempre maggiore di dati ha portato al crawling di internet. Questo ha comportato che porzioni di molti dataset NLP usati come benchmark venissero incluse nei dati di training.

Il rischio principale dovuto alla data contamination è il fatto che le valutazioni dei modelli potrebbero essere più elevate e non rispecchiare le capacità effettive.

Model	Accuracy on TQA-contaminated	Accuracy on TQA-uncontaminated
Gpt2-without-contamination	0.051471	0.050459
Gpt2-contaminated-with-answers-only	0.036765	0.016514
Gpt2-contaminated-with-questions-answer-pairs	0.143382	0.115596
Gpt2-contaminated-with-full-instructions	0.443015	0.275229

Fig. 1. Effetti della data contamination sull'accuratezza di alcuni modelli

La maggior parte dei lavori esistenti si è concentrata ad analizzare il problema della contaminazione nella fase di pre-training. Tuttavia, la fase di fine-tuning può anch'essa rappresentare un momento di contaminazione. In particolare, le metriche per la rilevazione di data contamination proposte si basano sull'analisi dei dati di pre-training attraverso tecniche di matching, sia a livello esatto che semantico. Tuttavia, questo approccio presuppone l'accesso completo ai corpora di addestramento, un requisito spesso non soddisfatto per modelli chiusi. Inoltre, molti LLM fine-tuned rilasciati dalla

comunità non forniscono report affidabili sulla contaminazione del dataset o non permettono l'accesso ai dati di fine-tuning. L'analisi dei modelli chiusi rappresenta una sfida ancora maggiore, ma è probabile che diventi la condizione prevalente nel prossimo futuro. Un'ulteriore criticità dei metodi open risiede nel loro elevato costo computazionale, che ne limita la scalabilità e l'applicabilità su larga scala.

II. BACKGROUND

A. *N*-gram

L'idea di base è confrontare le frasi presenti nel dataset di test con quelle usate per il pre-training e determinare se ci sono segmenti ripetuti tra i due. In altre parole, se il modello ha già visto una quantità significativa di testo del test set durante l'addestramento, la valutazione non rifletterà realmente la sua capacità di generalizzare a dati nuovi.

Per fare questo, si utilizza il concetto di **N-gramma**, ossia una sequenza contigua di N parole. Supponiamo di avere la frase:

“L'intelligenza artificiale sta rivoluzionando il mondo.”

Possiamo estrarre i seguenti *N*-grammi:

- Uni-grammi (1-gram): {"L'intelligenza", "artificiale", "sta", "rivoluzionando", "il", "mondo"}
- Bi-grammi (2-gram): {"L'intelligenza artificiale", "artificiale sta", "sta rivoluzionando", "rivoluzionando il", "il mondo"}
- Tri-grammi (3-gram): {"L'intelligenza artificiale sta", "artificiale sta rivoluzionando", "sta rivoluzionando il", "rivoluzionando il mondo"}

Se il dataset di pre-training contiene molti di questi *N*-grammi, allora possiamo sospettare una contaminazione.

Per rilevare la contaminazione, si seguono tre passaggi fondamentali:

- 1) **Creare un dizionario di N-grammi dal dataset di pre-training:** Durante la fase di pre-training, il modello viene esposto a un'enorme quantità di testi. Da questo corpus, possiamo estrarre tutti gli *N*-grammi e salvarli in una struttura dati efficiente (es. una tabella hash o un Bloom filter per risparmiare memoria).
- 2) **Estrarre gli N-grammi dal dataset di test:** Si effettua la stessa operazione sul dataset di test, generando un altro insieme di *N*-grammi.

- 3) **Confrontare i due insiemi:** Infine, calcoliamo la percentuale di N -grammi nel dataset di test che sono già presenti nel pre-training:

$$\text{Overlap}_N = \frac{|G_N(D_{\text{test}}) \cap G_N(D_{\text{pretrain}})|}{|G_N(D_{\text{test}})|} \quad (1)$$

dove:

- $|G_N(D_{\text{test}}) \cap G_N(D_{\text{pretrain}})|$ è il numero di N -grammi presenti in entrambi i dataset.
- $|G_N(D_{\text{test}})|$ è il numero totale di N -grammi nel dataset di test.

Se il valore di Overlap_N è alto, significa che il training set è contaminato e la valutazione del modello non è affidabile.

Per interpretare i risultati, si utilizza una soglia di riferimento. Se l'overlap supera un certo valore τ , si considera che il test set sia contaminato. Alcuni valori comuni sono:

- $\text{Overlap} < 0.05 \Rightarrow$ Nessuna contaminazione significativa.
- $0.05 < \text{Overlap} < 0.2 \Rightarrow$ Possibile contaminazione leggera.
- $0.2 < \text{Overlap} < 0.5 \Rightarrow$ Contaminazione moderata, la valutazione potrebbe essere falsata.
- $\text{Overlap} > 0.5 \Rightarrow$ Contaminazione grave, il test set deve essere sostituito.

Queste soglie dipendono dal dominio e dalla natura dei dati, ma forniscono un'idea generale.

L'uso dell' N -gram overlap presenta alcune limitazioni:

- **Sinonimi e Parafrasi**

Il metodo non riconosce frasi riformulate con parole diverse ma significato simile. Ad esempio:

- “L’IA sta cambiando il mondo.”
- “L’intelligenza artificiale sta trasformando la società.”

potrebbero non essere identificate come simili.

- **Scalabilità**

Confrontare milioni di N -grammi tra pre-training e test set è costoso in termini computazionali. Per ottimizzare l'operazione, si usano strutture dati efficienti come hashing locality-sensitive o Bloom filters.

- **Sensibilità alla scelta di N**

Se N è troppo piccolo, si rischiano falsi positivi (es. parole comuni). Se è troppo grande, si potrebbero perdere contaminazioni più sottili.

L' N -gram overlap è un metodo semplice e per questo motivo, le moderne tecniche di machine learning combinano l' N -gram overlap con altre metriche, come il Longest-Match o il ConTAM (Contamination Threshold Analysis Method), per ottenere un'analisi più robusta.

B. Embedding similarity search

L'Embedding Similarity Search è una tecnica che utilizza rappresentazioni vettoriali (embedding) di stringhe di testo, per confrontarne la vicinanza dei vettori in uno spazio numerico continuo, per identificarne la somiglianza. Questa metodologia è particolarmente utile rilevamento della contaminazione dei

dati. L'utilizzo di questa tecnica è sfruttata da diversi modelli utilizzati per il rilevamento della contaminazione accidentale nel pre-learning. Ad esempio, alcuni modelli che la utilizzano sono:

1) **BERT: BERT (Bidirectional Encoder Representations from Transformers)** è un modello particolarmente efficace nella generazione di **embedding contestuali** per i singoli token, grazie alla sua capacità di considerare l'intero contesto della frase durante l'elaborazione. Tuttavia, una sua limitazione fondamentale è l'assenza di un **embedding globale** che rappresenti l'intera frase in modo indipendente. Questo rende difficile confrontare frasi diverse all'interno di uno spazio vettoriale comune.

Per valutare la similarità semantica tra due frasi, per via dell'embedding su diversi spazi vettoriali, BERT utilizza un approccio **cross-encoder**, in cui entrambe le frasi vengono fornite simultaneamente al modello. L'output è un punteggio di similarità calcolato sulla base delle loro rappresentazioni congiunte. Questo metodo, tuttavia, è computazionalmente oneroso: dato un dataset di n frasi, confrontare ogni coppia richiede $\mathcal{O}(n^2)$ inferenze, rendendo l'approccio impraticabile per attività come la **ricerca semantica su larga scala** o il **clustering**.

Sebbene BERT sia un modello **bidirezionale**, poiché utilizza il meccanismo di **self-attention** per elaborare ogni token considerando il contesto circostante sia a sinistra che a destra, il motivo per cui necessita di entrambe le frasi in input per il confronto **non** è legato alla bidirezionalità. Piuttosto, dipende dalla sua architettura: BERT non genera embedding indipendenti per le frasi, ma elabora ciascuna coppia di frasi come un input unico, restituendo una rappresentazione che è specifica per quella coppia. Di conseguenza, ogni nuovo confronto richiede una nuova inferenza del modello, aumentando esponenzialmente il costo computazionale.

Questa caratteristica rende BERT **inefficiente e non scalabile** per applicazioni che richiedono il confronto rapido tra molteplici frasi, come la **semantic similarity search**. Per superare questa limitazione, è possibile utilizzare **SBERT (Sentence-BERT)** [5], che genera embedding indipendenti per ciascuna frase, consentendo di confrontarle direttamente mediante semplici metriche di similarità, come la **cosine similarity**, con una complessità significativamente ridotta ($\mathcal{O}(n)$ anziché $\mathcal{O}(n^2)$).

2) **SBERT:** Per risolvere questa inefficienza, SBERT (Sentence-BERT) [5] adotta un approccio siamese e triplet network, mappando ogni frase in un unico spazio vettoriale condiviso, indipendentemente dalle altre. Questo permette di confrontare frasi semplicemente calcolando la distanza tra i loro vettori embedding, senza dover rielaborare ogni coppia con il modello.

L'efficienza computazionale migliora drasticamente:

- Il tempo di ricerca della frase più simile passa da 65 ore a circa 2-3 millisecondi su gpu, poiché gli embedding possono essere calcolati in anticipo e confrontati in tempo reale con una semplice operazione di similarità

- La complessità si riduce da $\mathcal{O}(n^2)$ (BERT) a $\mathcal{O}(n)$ (SBERT), poiché ogni frase viene elaborata una sola volta per ottenere il suo embedding. E poi essendo sullo stesso spazio vettoriale si può calcolare una ricerca matematica tramite il cosine similarity.

Un aspetto fondamentale di SBERT è la trasformazione delle frasi in vettori di dimensione fissa, indipendentemente dalla lunghezza della frase. Per ottenere ciò, SBERT applica un livello di pooling sull'output di BERT, aggregando le rappresentazioni contestuali di ogni token in un unico vettore rappresentativo. Esistono tre strategie principali:

- 1) **CLS Pooling**: Si utilizza direttamente il vettore associato al token [CLS], che rappresenta l'intera sequenza. Tuttavia, BERT non è stato addestrato esplicitamente per assegnare significato semantico forte a questo token, rendendolo meno efficace.
- 2) **Mean Pooling** (default) : Si calcola la media dei vettori di tutti i token nella frase, ottenendo una rappresentazione più stabile. Questo metodo è spesso il più efficace
- 3) **Max Pooling**: Si seleziona, per ogni dimensione del vettore, il valore massimo tra tutti i token. Questo enfatizza le informazioni più rilevanti, ma può essere più sensibile al rumore

L'uso del pooling è un metodo per trasformare un insieme di vettori (uno per ogni token) in un singolo vettore che rappresenta l'intera frase. Inoltre, gli embedding ottenuti vengono poi confrontati tra loro usando cosine similarity, che è un'operazione matematica estremamente veloce e non richiede di passare nuovamente nel modello. Essa misura l'angolo tra i vettori nel loro spazio vettoriale, ignorando la loro magnitudine:

$$\text{cosine-sim}(u, v) = \frac{u \cdot v}{||u|| ||v||}$$

Questa metrica ha diversi vantaggi:

- È indipendente dalla lunghezza dei vettori
- Fornisce valori tra -1 (totalmente diversi) e 1 (identici)
- È computazionalmente efficiente, essendo un'operazione semplice tra vettori normalizzati

Quindi più la misura è prossima all'unità e più le loro semantiche sono simili. Nel caso specifico di data contamination accidentale nel pre-learning, quello che si fa è:

- Creare embedding delle frasi nei dati di test con SBERT
- Confrontarli con i dati di addestramento usando la cosine similarity
- Applicare una soglia (es. 0.7): se due frasi hanno similarità superiore a questa soglia, è probabile che siano state viste in fase di addestramento, indicando contaminazione accidentale. Questo perché non essendo una soglia elevata significa non sono identiche le semantiche, come in un caso di contaminazione volontaria.

Questa tecnica permette di individuare contaminazioni nei dataset in modo scalabile ed efficiente.

Molto importante sottolineare che SBERT cattura la semantica

della frase nel suo insieme, considerando la semantica completa della frase tramite una rete Transformer. Questo lo rende molto più efficace nel confrontare frasi basandosi sul loro significato.

C. Influence function

L'influence function è una strategia efficace per individuare la contaminazione del dataset e può essere applicata quando sono disponibili sia il modello addestrato sia i dati di training. Il suo obiettivo principale, come specificato nel paper [2], è misurare in che modo ciascun esempio di training influenza le predizioni del modello, permettendo così di rilevare eventuali sovrapposizioni tra i dati di addestramento e quelli di test.

Il metodo si basa sull'analisi dell'impatto dei singoli campioni di training rispetto a un determinato campione di test. Per ciascun esempio del test set, viene calcolato un **fattore di influenza**, che quantifica il contributo di ogni campione di training sulla predizione del modello per quel dato specifico. Questo calcolo viene effettuato iterativamente su tutti i campioni di training, stimando come cambierebbe la predizione se un certo esempio fosse rimosso dal dataset di addestramento.

Una volta determinati i valori di influenza, i campioni di training vengono ordinati in base alla loro rilevanza rispetto al campione di test analizzato. I più influenti vengono poi esaminati per verificare la presenza di eventuali casi di contaminazione, ossia situazioni in cui il modello potrebbe aver già visto informazioni troppo simili a quelle presenti nei dati di test. Questo fenomeno può portare a una sovrastima delle prestazioni del modello, compromettendo la sua capacità di generalizzare correttamente a dati nuovi.

Sebbene l'influence function sia uno strumento potente per diagnosticare la contaminazione del dataset, il suo utilizzo comporta un elevato costo computazionale, nell'ordine di $\mathcal{O}(M^2 + MN)$, dove M denota la dimensione del training set e N indica la dimensione del test set. Il processo richiede infatti il calcolo iterativo dell'influenza per un grande numero di campioni di training, rendendolo un'operazione particolarmente onerosa per modelli di grandi dimensioni. Per questa ragione, vengono spesso implementate ottimizzazioni o tecniche di approssimazione per ridurre il tempo di calcolo, mantenendo comunque l'efficacia del metodo nell'identificare eventuali anomalie nei dati.

1) *Formalizzazione matematica*: Prima di poter enunciare la definizione matematica di influence function, è opportuno fare un riferimento al paper [1] per poter introdurre il contesto teorico e le nozioni fondamentali su cui si basa.

Sia $f_\theta : X \rightarrow Y$ il processo di predizione di un modello linguistico, dove X rappresenta lo spazio di input e Y denota lo spazio di target; il modello f è parametrizzato da θ . Dato un dataset di training $\mathcal{D} = \{\{z_i = (x_i, y_i)\}, \forall i = 1, \dots, N\}$ e uno spazio dei parametri Θ , consideriamo il minimizzatore di rischio empirico come:

$$\theta^* = \arg \min_{\theta \in \Theta} \frac{1}{N} \sum_{i=1}^N \mathcal{L}(z_i, \theta)$$

dove $\mathcal{L}$ è la loss function e f_{θ^*} è convergente a θ^* .

L'influence function stabilisce un framework statistico rigoroso per quantificare l'impatto di un singolo dato di training sul modello dell'output. Descrive il grado di variazione dei parametri del modello quando si perturba un determinato campione di addestramento. In modo più specifico, consideriamo la seguente relazione di aggiornamento del peso per osservare come cambiano i parametri appresi:

$$\theta_{\varepsilon,k} = \arg \min_{\theta \in \Theta} \frac{1}{N} \sum_{i=1}^N \mathcal{L}(z_i, \theta) + \varepsilon \mathcal{L}(z_k, \theta)$$

dove z_k è il k -esimo sample nel training set. L'influenza del data point $z_k \in \mathcal{D}$ sul minimizzatore di rischio empirico θ^* definito come la derivata di $\theta_{\varepsilon,k}$ per $\varepsilon = 0$:

$$\mathcal{I}_{\theta^*}(z_k) = \left. \frac{d\theta_{\varepsilon,k}}{d\varepsilon} \right|_{\varepsilon=0} \approx -H_{\theta^*}^{-1} \nabla_{\theta} \mathcal{L}(z_k, \theta^*)$$

dove $H_{\theta^*} = \nabla_{\theta}^2 \frac{1}{N} \sum_{i=1}^N \mathcal{L}(z_i, \theta^*)$ è l'Hessiana della loss function.

Ovviamente affinché la matrice Hessiana possa esistere viene assunto che il rischio empirico sia due volte differenziabile e fortemente convesso in θ . Nel caso in cui il modello non sia convergente oppure lavora con funzioni obiettivo non convesse, allora l'Hessiana potrebbe avere autovalori negativo o essere non invertibile. Per risolvere questo problema si applica una tecnica di "damping", ovvero, $H_{\theta^*} \leftarrow H_{\theta^*} + \lambda I$, per rendere la matrice Hessiana definita positiva così da assicurarne l'invertibilità. Quindi secondo la chain rule, l'influenza di z_k sulla loss in un test point z_{test} ha la seguente espressione in forma chiusa:

$$\mathcal{I}(z_{\text{test}}, z_k) = -\nabla_{\theta} \mathcal{L}(z_{\text{test}}, \theta^*)^{\top} H_{\theta^*}^{-1} \nabla_{\theta} \mathcal{L}(z_k, \theta^*)$$

Ad alto livello, l'influence function $\mathcal{I}(z_{\text{test}}, z_k)$ misura l'impatto di un training data point z_k sul test sample z basato sui parametri cambiati del modello. Quindi una grande influenza significa grandi cambiamenti nei parametri quando si perturba il valore di z_k . In questo modo la funzione misura in modo intuitivo il contributo di z_k su z_{test} .

2) *Influence function negli LLM*: Molti LLM vengono pre-addestrati utilizzando la cross-entropy, che è due volte differenziabile e fortemente convessa, permettendo di applicare direttamente le funzioni di influenza per misurare l'impatto di ciascun campione di training su un punto di validazione. Tuttavia, a causa dell'enorme quantità di dati e parametri, il calcolo esatto di *iHVP* (inversa del prodotto vettore-Hessiano) per un intero LLM è computazionalmente proibitivo, che rappresenta la causa principale del costo oneroso di questa tecnica.

La influence function viene applicata nella fase di pre-training e non nella fase di fine-tuning per i seguenti motivi:

- Durante il fine-tuning, il modello viene adattato a dati specifici con un sottoinsieme più piccolo di parametri aggiornabili (es. parameter-efficient fine-tuning come LoRA, Adapters). Ciò rende il calcolo dell'Hessiana meno stabile e meno rappresentativo dell'intero modello, compromettendo la validità dell'influence function, mentre, nella fase di pre-training, il modello viene addestrato

su un dataset statico e ben definito, rendendo più praticabile il calcolo dell'influenza di ogni campione di training

- Studi recenti hanno mostrato che il calcolo di *iHVP* per LLM è già difficile durante il pre-training, ma diventa ancora più instabile e meno informativo nel fine-tuning. Questo perché le funzioni di influenza si basano sull'ottimizzazione globale del modello, mentre il fine-tuning introduce cambiamenti locali che non sempre seguono la stessa dinamica del pre-training

D. Decoding matching

Questo metodo può essere utilizzato come alternativa nei casi in cui i dati di training non sono accessibili mentre il modello è accessibile. In questo caso quello che si vuole fare è sfruttare la *memorizzazione* del modello. La definizione formale di memorizzazione può essere trovata a *Carlini et al. (2021,2022)* [3]. Un modello θ conosce una stringa s se s può essere estratta interagendo con il modello. In particolare, ci concentriamo sull'interazione black-box dove il modello genera s come la continuazione più probabile dato un prompt c . Passiamo ora a definire la perplessità. Data una sequenza d di lunghezza N , la perplessità è calcolata come

$$P(d) = -\frac{1}{N} \sum_{i=1}^N \log_2 q(d_i | d_{<i})$$

dove $q(d_i | d_{<i})$ rappresenta la probabilità condizionale di d_i dati i token precedenti $d_{<i}$.

Una sequenza è considerata memorizzata se il modello ha una perplessità bassa su essa. L'idea è che le sequenze del test set presenti nel training set avranno una perplessità notevolmente più alta rispetto a quelle che non sono mai state mai viste. Computare semplicemente la perplessità di un'istanza non ci fornisce informazioni sulla contaminazione. A tal fine è necessario definire una *memorised baseline* e una *clean baseline*. La memorised baseline viene calcolata sui dati che sono sicuramente nel training set. La clean baseline, invece, viene calcolata sui dati che non sono nel training set (ad esempio quelli che seguono la data di creazione del modello). A seconda della vicinanza ad una o l'altra baseline possiamo identificare se il modello esibisce memorizzazione sul test set. Per rendere il paragone tra le due baseline *fair* è necessario che i dati su cui vengono valutate le baseline provengano dalla stessa fonte e abbiano la stessa struttura. Nel paper di riferimento si considerano i dataset che prendono come fonte gli articoli di Wikipedia (ad esempio SQuAD). Un'alternativa è sfruttare la definizione di sequenza *estraibile* sempre presente nel paper [3]. Una sequenza è estraibile da un LM f_{θ} se esiste un prefisso c tale che:

$$s \leftarrow \arg \max_{S': |S'|=N} f_{\theta}(s' | c)$$

dove con un abuso di notazione $f_{\theta}(s' | c)$ indica la likelihood della sequenza s' . La percentuale di contaminazione può essere calcolata come il numero di esempi estraibili dal modello diviso il numero di esempi.

memorised Kanye Omari West (/kænj/; born June 8, 1977) is an American rapper, singer, songwriter, record producer, entrepreneur, and fashion designer.[2] His music has spanned a broad range of styles, incorporating an eclectic range of influences including hip hop, soul, baroque pop, electro, indie rock, synth-pop, industrial, and gospel. Born in Atlanta and raised in Chicago, West first became known as a producer for Roc-A-Fella Records in the early 2000s, producing singles for recording artists such as Jay-Z, Ludacris, and Alicia Keys. Intent on pursuing a solo career as a rapper, West released his debut album <i>The College Dropout</i> in 2004 to critical and commercial success, and founded the record label GOOD Music. He went on to experiment with a variety of musical genres on subsequent acclaimed studio albums, including <i>Late Registration</i> (2005), <i>Graduation</i> (2007), and the polarizing but influential <i>808s & Heartbreak</i> (2008). Released in 2010, West's fifth album <i>My Beautiful Dark Twisted Fantasy</i> became his most critically acclaimed album and is widely regarded by critics as the best album of the 2010s.[3] He has since succeeded the album with <i>Yeezus</i> (2013), <i>The Life of Pablo</i> (2016), <i>Ye</i> (2018), and <i>Jesus Is King</i> (2019), as well as full-length collaborations <i>Watch the Throne</i> (2011) and <i>Kids See Ghosts</i> (2018) with Jay-Z and Kid Cudi respectively.	clean Yéǝǝǝ (/jɛ/; YAY; born Kanye Omari West /kænj/; KAHN-vec; June 8, 1977) is an American rapper, singer, songwriter, record producer, and fashion designer. Born in Atlanta and raised in Chicago, West gained recognition as a producer for Roc-A-Fella Records in the early 2000s, producing singles for several prominent artists and developing a production style that heavily sampled old-school soul music. Intent on pursuing a solo career as a rapper, he released his debut studio album, <i>The College Dropout</i> (2004), to critical and commercial success. He founded the record label GOOD Music later that year. West's subsequent albums have seen continued commercial and critical success: he graduated to widespread critical acclaim with his second album, <i>Late Registration</i> (2005), the tandem release of <i>Graduation</i> (2007) and <i>808s & Heartbreak</i> (2008), and the polarizing, but influential, <i>My Beautiful Dark Twisted Fantasy</i> (2010). Tracks such as "Diamonds from Sierra Leone" and "Stronger" reached number one on the Billboard Hot 100, West released his abrasive sixth album, <i>Yeezus</i>, to further critical praise in 2013. In 2016, he released his seventh album, <i>The Life of Pablo</i>, a wide departure from the brash hip-hop style that had characterized his earlier work. Although it extensively used the auto-tune audio effect, the album was still critically acclaimed. Within the first week of release, it became West's second consecutive number one album on the Billboard
---	--

Fig. 2. Esempio di memorizzazione di GPT-3 tra *memorised baseline* e *clean baseline*.

III. ESTENSIONE DELLE METRICHE APPLICATE AL FINE-TUNING

Tutte le metriche analizzate, quando applicate nella fase di pre-training, si basano sulla selezione di una soglia (*threshold*) che determina il grado di contaminazione dell'input analizzato. Per estendere queste tecniche alla fase di fine-tuning, con l'obiettivo di rilevare casi di contaminazione intenzionale, è possibile innalzare il valore della soglia fino a un valore prossimo a 1.

In questo modo, il modello sarà in grado di identificare con maggiore precisione gli input che presentano una semantica identica, riducendo il rischio di falsi positivi derivanti da semplici somiglianze lessicali o parafrasi. Questo approccio consente di adattare i metodi di rilevamento della contaminazione anche a contesti in cui la sovrapposizione tra dati di addestramento e dati di test è stata introdotta deliberatamente, garantendo un'analisi più rigorosa dell'effettiva capacità di generalizzazione del modello.

IV. SOLUZIONE PROPOSTA

Le soluzioni analizzate finora, quali N-gram e Embedding Similarity Search, non si sono rivelate particolarmente efficaci nel riconoscere frasi riformulate che mantengono lo stesso significato semantico. Questa limitazione è particolarmente evidente nel caso degli N-gram, che risultano poco adatti a catturare variazioni nella formulazione delle frasi. Per quanto riguarda la metrica basata sulla similarità tra embedding, essa si dimostra eccessivamente sensibile alla soglia scelta, con il rischio di generare un numero elevato di falsi positivi o falsi negativi, compromettendo così l'affidabilità della valutazione. La nostra proposta prevede la creazione di un **regressore** che, a partire da un training set ed un test set, restituisca il livello di data contamination. Durante l'addestramento si prende ispirazione dal modello GAN. L'idea è quella di avere un generatore che generi istanze sintetiche a partire da quelle originali (che altro non sono che riscritture che conservano la stessa semantica delle frasi originali) ed un discriminatore. Nel dettaglio, la soluzione si compone dei seguenti attori:

- Dataset di partenza.
- Generatore di istanze sintetiche. Il generatore prende in input k istanze dal dataset originale (il valore k viene modificato ad ogni epoca di addestramento) e del

rumore z . Quello che fa è generare un'istanza sintetica $z' = G(x, z)$ che vogliamo essere la stessa frase ma refrasata.

- Discriminatore: fa due operazioni. Classifica le istanze sintetiche e le istanze non sintetiche. Fornisce in output la percentuale di contaminazione (sappiamo che il valore target è $\frac{k}{k+n}$, dove n è la cardinalità del dataset).

V. DISCUSSIONE E POSSIBILE IMPLEMENTAZIONE

Andiamo ad analizzare le componenti della loss function nel dettaglio. Il discriminatore utilizza una loss function composta da due parti. Per la classificazione si usa la **binary cross-entropy loss**:

$$\mathcal{L}_{CE} = -\mathbb{E}_{x \sim p_{data}} [\log D(x)] - \mathbb{E}_{(x,z) \sim p_{data}, p_z} [1 - \log D(G(x, z))]$$

Per la regressione si utilizza MSE sulla percentuale predetta. Da notare che è importante che k vari di epoca in epoca altrimenti il discriminatore predirebbe banalmente sempre la stessa percentuale:

$$\mathcal{L}_{MSE} = \frac{1}{2} \left\| \frac{k}{n+k} - y \right\|^2$$

Dunque la funzione di perdita finale è:

$$\mathcal{L}_D = \mathcal{L}_{CE} + \mathcal{L}_{MSE}$$

Per quanto riguarda il generatore, invece, si compone di un **adversarial loss** (per cercare di imbrogliare il classificatore)

$$\mathbb{E}_{(x,z) \sim p_{data}, p_z} [1 - \log D(G(x, z))]$$

e di una **semantic consistency loss** per far sì che quanto generato sia una riscrittura dell'input originale

$$\mathcal{L}_{semantic} = 1 - Sim(x, G(x, z))$$

In questo caso Sim è una qualsiasi misura di similarità tra due embeddings. Quindi per il generatore finale abbiamo:

$$\mathcal{L}_G = \lambda_1 \mathbb{E}_{(x,z) \sim p_{data}, p_z} [1 - \log D(G(x, z))] + \lambda_2 \mathcal{L}_{semantic}$$

Durante l'addestramento si vuole quindi fare la seguente minimizzazione:

$$\min_{G,D} V(D, G) = \min_{G,D} \mathcal{L}_D + \mathcal{L}_G$$

Una volta che il modello è stato addestrato si considera solo il discriminatore. Ad esso occorre fornire in input l'unione tra il dataset di training ed il dataset di testing e restituirà la percentuale di data contamination.

REFERENCES

- [1] Autori anonimi (paper sotto revisione per la ICLR 2025). *Do Influence Functions work on Large Language Models?* 2025.
- [2] S. Yang, W-L. Chiang, L. Zheng, J. E. Gonzalez, I. Stoica. *Rethinking Benchmark and Contamination for Language Models with Rephrased Samples*. In *Proceedings of the 2023 Conference on Neural Information Processing Systems (NeurIPS 2023)*, 2023.

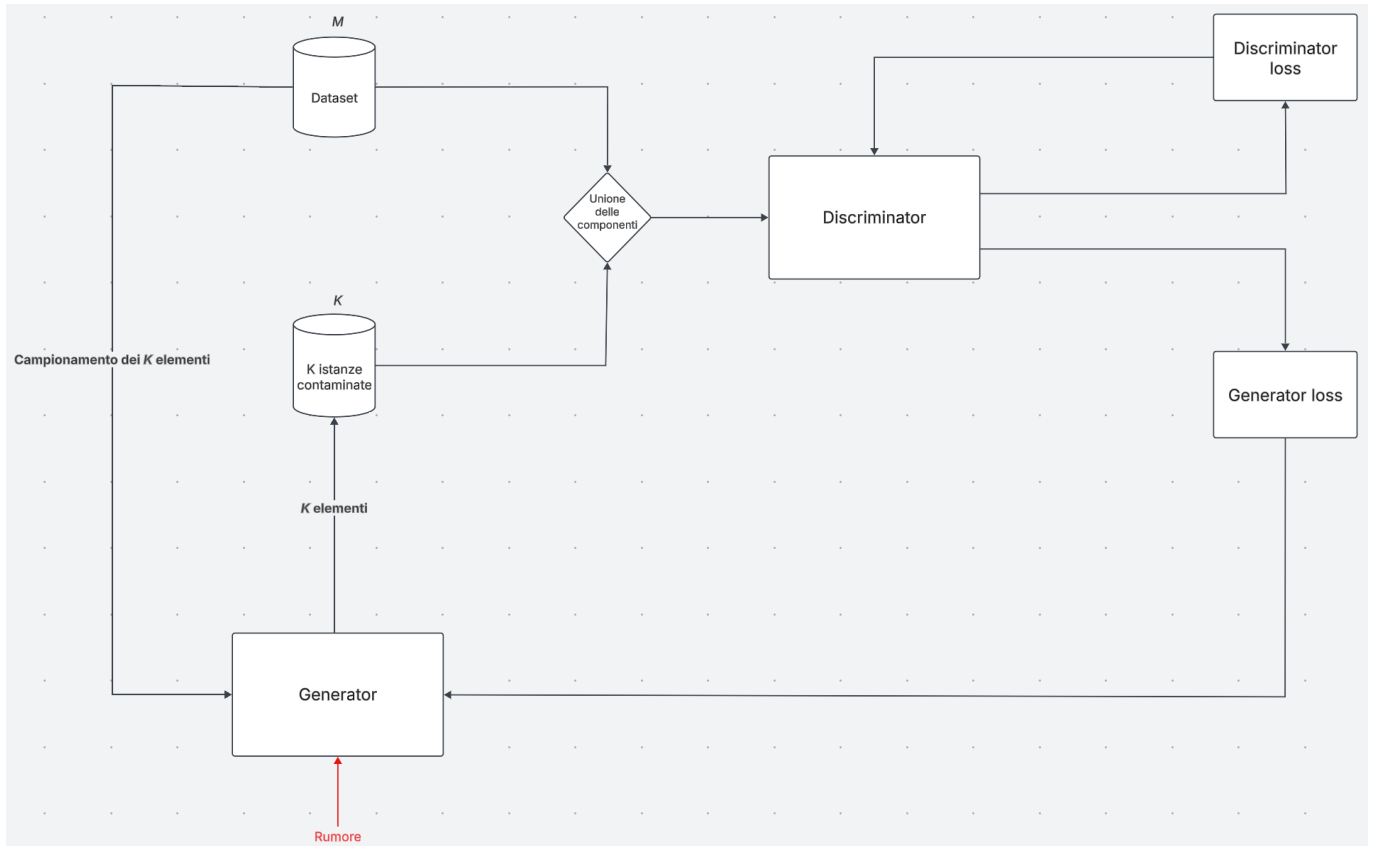


Fig. 3. Struttura della GAN.

- [3] N. Carlini, F. Tramer, E. Wallace, M. Jagielski, A. Herbert-Voss, K. Lee, A. Roberts, T. Brown, D. Song, U. Erlingsson, et al. *Extracting training data from large language models*. In *30th USENIX Security Symposium (USENIX Security 21)*, pages 2633–2650, 2021.
- [4] L. Wang, N. Yang, X. Huang, B. Jiao, L. Yang, D. Jiang, R. Majumder, F. Wei. *Text Embeddings by Weakly-Supervised Contrastive Pre-training*. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing (EMNLP 2024)*, 2024.
- [5] N. Reimers and I. Gurevych. *Sentence Embeddings using Siamese BERT-Networks*. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP 2019)*, pages 3980–3990, 2019.
- [6] A. Smith, B. Johnson, C. Lee, D. Kim. *Estimating Contamination via Perplexity*. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP 2023)*, 2023.
- [7] D. Jurafsky, J. H. Martin. *N-gram Language Models*. In *Speech and Language Processing (2nd Edition)*. Prentice Hall, 2009.
- [8] M. Zou, Y. Xu, J. Wei, D. Chen, S. Gururangan. *Towards Data Contamination Detection for Modern Large Language Models*. In *Proceedings of the 2024 Conference on Neural Information Processing Systems (NeurIPS 2024)*, 2024.
- [9] T. Lin, P. Shaw, S. Raffel, K. Lee. *Evaluation Data Contamination in LLMs: How Do We Measure It and (When) Does It Matter?*. In *Proceedings of the 2024 International Conference on Learning Representations (ICLR 2024)*, 2024.
- [10] D. Jurafsky, J. H. Martin. *Speech and Language Processing (3rd Edition, draft)*. Online manuscript, 2023. URL: <https://web.stanford.edu/~jurafsky/slp3/>.
- [11] J. Kaplan, S. McCandlish, T. Henighan, T. B. Brown, B. Chess, R. Child, S. Gray, A. Radford, J. Wu, D. Amodei. *Scaling Laws for Neural Language Models*. In *arXiv preprint arXiv:2001.08361*, 2020.
- [12] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, et al. *Language Models are Few-Shot Learners*. In *Advances in Neural Information Processing Systems (NeurIPS 2020)*, 2020.
- [13] J. Chowdhery, S. Narang, J. Devlin, M. Bosma, G. Mishra, A. Roberts, P. Barham, H. Chung, C. Sutton, S. Gehrmann, et al. *PaLM: Scaling Language Modeling with Pathways*. In *Proceedings of the 2022 International Conference on Machine Learning (ICML 2022)*, 2022.